- **Manifesto para o Desenvolvimento Ágil de Software (Site Oficial):** Manifesto Ágil - Declaração Oficial (Português)

- **Os 12 Princípios do Manifesto Ágil (Site Oficial):** Os Doze Princípios do Software Ágil (Português)

- **História do Encontro de Snowbird (Site Oficial):** History: The Agile Manifesto

- **Engenharia de Software Moderna (Prof. Marco Tulio Valente - UFMG):** Capítulo 2: Processos Ágeis

3- Introdução ao Scrum



Este diagrama apresenta a visão geral do **Ciclo Scrum** e como as etapas do processo se conectam com seus fundamentos empíricos:

- **Entradas e Planejamento:** Organização e ordenação de prioridades no *Product Backlog*, seguido da escolha dos itens no *Sprint Planning* para definir o *Sprint Backlog*.

- **Execução e Acompanhamento:** Desenvolvimento ao longo da *Sprint* com reuniões diárias (*Daily Scrum*) de 15 minutos para alinhar o progresso e remover impedimentos.

- **Entrega e Aprendizado:** Validação do *Incremento* funcional na *Sprint Review* com os stakeholders e identificação de melhorias do processo na *Sprint Retrospective*.

Exemplos dos 3 Pilares:

- **Transparência:** Manter o Quadro Kanban visível para toda a equipe e stakeholders.
- **Inspeção:** Avaliar o progresso diariamente no *Daily Scrum* e na *Sprint Review*.
- **Adaptação:** Corrigir falhas e ajustar a rotina de trabalho durante a *Sprint Retrospective*.

Capítulo 3 – O que é Scrum

Agora que você já sabe qual problema o Scrum tenta resolver e de que movimento ele faz parte, podemos olhar para ele diretamente. Neste capítulo, vamos entender a definição oficial, conhecer a história do Scrum e ver por que ele se tornou tão importante no desenvolvimento de software.

3.1 A definição oficial, palavra por palavra

O Scrum Guide define o Scrum como um **framework leve que ajuda pessoas, times e organizações a gerar valor por meio de soluções adaptativas para problemas complexos**. Essa frase é curta, mas cada pedaço dela carrega uma ideia importante, então vale a pena desmontá-la.

Framework: como vimos no capítulo anterior, é uma estrutura que define os elementos essenciais e as regras entre eles, sem detalhar todas as técnicas. O Scrum diz que haverá uma Sprint, um Product Owner, um Product Backlog e assim por diante, mas não diz como você deve programar, testar ou desenhar telas.

Leve: o Scrum tem poucos elementos. O Scrum Guide inteiro cabe em menos de quinze páginas. Isso é proposital. Quanto menos regras um framework tem, menos ele atrapalha a equipe e mais espaço sobra para que ela descubra o que funciona no seu contexto.

Pessoas, times e organizações: o Scrum não é só para programadores. Ele envolve a equipe que faz o trabalho, mas também as pessoas interessadas no produto e a organização como um todo. Uma empresa que adota Scrum só no time de desenvolvimento, mas mantém o resto da organização funcionando de forma rígida, costuma ter dificuldades.

Gerar valor: o objetivo não é “cumprir tarefas” nem “seguir o processo”, e sim produzir algo que tenha valor para alguém. Valor pode ser um usuário conseguindo pedir comida mais rápido, um restaurante vendendo mais ou uma empresa reduzindo custos. Uma equipe que entrega muitas funcionalidades inúteis está ocupada, mas não está gerando valor.

Soluções adaptativas para problemas complexos: retoma tudo o que vimos no Capítulo 1. Como o problema é complexo, a solução não pode ser fixada no início; ela precisa se adaptar à medida que aprendemos.

Logo após a definição, o Scrum Guide resume o funcionamento do Scrum em quatro passos, num ambiente que o Scrum Master ajuda a criar. Em linguagem simples, é assim: primeiro, o **Product Owner** organiza o trabalho necessário para resolver um problema complexo em uma lista chamada **Product Backlog**. Segundo, o **Scrum Team** transforma uma parte desse trabalho em um **Increment** de valor durante uma **Sprint**. Terceiro, o Scrum Team e as partes interessadas inspecionam os resultados e fazem ajustes para a próxima Sprint. Quarto, o ciclo se repete.

Não se preocupe se esses termos ainda parecem estranhos. Cada um deles terá uma seção própria mais adiante. Por enquanto, guarde a ideia central: existe uma lista ordenada do que precisa ser feito, a

equipe pega uma parte dessa lista, transforma em algo utilizável dentro de um ciclo curto, todos olham o resultado, ajustam o rumo e começam outro ciclo.

O Scrum Guide acrescenta mais duas observações que você deve guardar. A primeira é que o Scrum é **propositalmente incompleto**: ele define apenas as partes necessárias para aplicar a teoria do Scrum, e a inteligência coletiva das pessoas que o usam completa o restante. A segunda é que os elementos do Scrum (papéis, eventos, artefatos e regras) são **imutáveis**. Isso não significa que a equipe não possa adicionar práticas; significa que, se você retirar ou alterar partes essenciais, o resultado pode até funcionar, mas não é mais Scrum. O guia diz que o Scrum só existe em sua totalidade. Por exemplo, uma equipe que trabalha em ciclos de duas semanas, mas nunca faz a Retrospective e não tem um Product Owner, está usando algumas ideias do Scrum, mas não está usando Scrum.

3.2 De onde vem o nome e como o Scrum surgiu

O nome Scrum vem do **rugby**. No rugby, o *scrum* é uma formação em que os jogadores dos dois times se agrupam, abraçados e empurrando juntos, para disputar a posse da bola. A imagem transmite a ideia de um time inteiro se movendo como uma unidade, em vez de cada jogador fazer sua parte isoladamente.

A origem dessa metáfora está num artigo publicado em 1986 na revista Harvard Business Review pelos pesquisadores japoneses **Hirotaka Takeuchi e Ikujiro Nonaka**, intitulado *The New New Product Development Game*. Eles estudaram empresas que desenvolviam produtos inovadores com rapidez, como fabricantes de câmeras, copiadoras e automóveis, e perceberam que elas não trabalhavam como numa corrida de revezamento, em que um corredor entrega o bastão ao próximo (exatamente como as fases da cascata). Em vez disso, essas empresas usavam equipes pequenas e multidisciplinares, que trabalhavam juntas do início ao fim, sobrepondo as fases e passando a bola de um lado para o outro, como no rugby. Os autores usaram a palavra *scrum* para descrever esse estilo.

No início dos anos 1990, **Jeff Sutherland**, que trabalhava com desenvolvimento de software nos Estados Unidos, se inspirou nesse artigo e em outras ideias para organizar uma equipe de forma diferente, em ciclos curtos e com o time trabalhando junto. Na mesma época, **Ken Schwaber** vinha estudando por que os processos tradicionais de software falhavam e chegou a conclusões parecidas, apoiado na ideia de controle empírico de processos, que veremos no próximo capítulo. Os dois passaram a colaborar e, em **1995**, apresentaram juntos o Scrum pela primeira vez numa conferência de software chamada OOPSLA. Em 2001, ambos estavam entre os signatários do Manifesto Ágil, e o Scrum passou a ser visto como uma das principais formas de colocar os valores ágeis em prática.

3.3 O Scrum Guide e sua evolução

Durante muitos anos, o Scrum foi transmitido por livros, cursos e artigos, e cada autor descrevia o framework de um jeito um pouco diferente. Para criar uma referência única, Schwaber e Sutherland escreveram em **2010** a primeira versão do **Scrum Guide**, que eles chamam de “o guia definitivo do Scrum” ou “as regras do jogo”. O guia é gratuito e está disponível em diversos idiomas, inclusive em português, no site scrumguides.org.

O guia foi atualizado algumas vezes desde então: em 2011, 2013, 2016 (quando foram incluídos os cinco valores do Scrum), 2017 e **2020**. A versão de 2020 é a atual e oficial, e trouxe mudanças importantes que você vai encontrar ao longo desta apostila. Entre elas, o guia ficou menos prescritivo, ou seja, passou a dar menos receitas prontas e mais espaço para a equipe decidir; deixou de usar linguagem específica de TI, para facilitar o uso do Scrum em outras áreas; trocou a ideia de um "Time de Desenvolvimento" separado dentro do Scrum Team pela ideia de **um único time**, no qual as pessoas que constroem o produto são chamadas de **Developers**; introduziu o **Product Goal** (o objetivo de longo prazo do produto); e associou a cada artefato um **compromisso**, assunto do Capítulo 7.

Essa história ajuda a entender um fenômeno comum: muitos materiais na internet, e até em empresas, ainda descrevem o Scrum como ele era em 2013 ou 2017. Por isso você pode encontrar referências a "Time de Desenvolvimento", às "três perguntas da Daily" ou à ideia de que o Scrum Master "remove impedimentos" pessoalmente. Quando houver divergência, a referência válida é o Scrum Guide de 2020.

3.4 Por que o Scrum é tão usado no desenvolvimento de software

O Scrum Guide deixa claro que o Scrum pode ser usado em vários tipos de trabalho, não só em software. Ainda assim, foi no desenvolvimento de software que ele nasceu e onde ele é mais difundido, e pesquisas de mercado apontam há anos o Scrum como o framework ágil mais adotado. Há boas razões para isso.

A primeira razão é que o Scrum **reduz o risco**. Como a equipe entrega algo utilizável a cada ciclo de no máximo um mês, o pior cenário possível é desperdiçar um ciclo, e não um ano inteiro. Se a equipe do aplicativo de pedidos descobre na terceira Sprint que os usuários quase não usam a função de "favoritar pratos", ela pode parar de investir nisso imediatamente.

A segunda razão é que o Scrum **traz o valor mais cedo**. Como o trabalho é ordenado pelo valor, as funcionalidades mais importantes tendem a ficar prontas primeiro. Às vezes, um produto com 30% das funcionalidades planejadas já resolve 80% do problema do cliente e pode ser lançado.

A terceira razão é a **transparência**. No Scrum, todos podem ver o que está sendo feito, o que já está pronto e quais problemas surgiram. Isso evita a situação clássica em que o projeto parece "90% pronto" por meses a fio.

A quarta razão é a **capacidade de adaptação**. Mercado, tecnologia e necessidades mudam, e o Scrum oferece momentos regulares para incorporar essas mudanças sem bagunçar o trabalho em andamento.

A quinta razão está nas **pessoas**. O Scrum dá à equipe autonomia para decidir como fazer o trabalho, o que costuma aumentar o engajamento e a qualidade.

É justo, porém, terminar este capítulo com um alerta. O Scrum não é uma fórmula mágica. Ele não resolve sozinho problemas de falta de habilidade técnica, de liderança autoritária ou de metas impossíveis. O que ele faz muito bem é **tornar esses problemas visíveis** rapidamente, para que possam ser tratados. Se a equipe nunca consegue terminar o que planeja, o Scrum vai mostrar isso a cada Sprint. Cabe às pessoas usar essa informação para melhorar. E essa ideia de tornar as coisas visíveis para poder melhorá-las é exatamente o fundamento teórico do Scrum, que estudaremos a seguir.

Os fundamentos do Scrum

Muita gente aprende Scrum decorando a lista de papéis, eventos e artefatos. Isso funciona para passar numa prova rápida, mas não ajuda a usar o Scrum de verdade. Os elementos do Scrum só fazem sentido quando entendemos a teoria por trás deles. Este capítulo trata justamente dessa teoria: o empirismo, o pensamento Lean, os três pilares e os cinco valores. Se você entender bem este capítulo, vai perceber que os capítulos seguintes são apenas a aplicação prática dessas ideias.

4.1 Empirismo e pensamento Lean

O Scrum Guide afirma que o Scrum se baseia em duas linhas de pensamento: o **empirismo** e o **pensamento Lean**.

A palavra empirismo vem da filosofia e tem a ver com experiência. O empirismo afirma que **o conhecimento vem da experiência** e que as **decisões devem ser tomadas com base no que é observado**. Parece óbvio, mas compare com a cascata: nela, as decisões mais importantes (o que construir, como, em quanto tempo) são tomadas no início, quando ainda não há experiência nenhuma com aquele produto. No empirismo, a equipe faz uma pequena parte, observa o resultado e só então decide o próximo passo com base em fatos, não em suposições.

Um exemplo do cotidiano ajuda. Imagine que você vai cozinhar um prato que nunca fez. Uma abordagem é seguir a receita à risca e só provar no final. Outra é ir provando enquanto cozinha: está sem sal, adiciona um pouco; ficou grosso demais, acrescenta água. A segunda abordagem é empírica. Você toma decisões a partir do que observa, e o resultado tende a ser melhor, principalmente quando a receita é nova para você.

No desenvolvimento de software, o equivalente a "provar a comida" é colocar uma versão funcionando nas mãos de alguém e observar o que acontece. A equipe do aplicativo de pedidos pode achar que os usuários vão adorar filtrar restaurantes por tempo de entrega. Em vez de apostar meses nisso, ela entrega uma versão simples do filtro e observa os dados de uso. Se quase ninguém usa, a decisão de investir mais nessa funcionalidade passa a ser baseada em evidência.

Para isso funcionar, o Scrum usa uma abordagem **iterativa e incremental**, termos que você já conhece do Capítulo 2. Segundo o Scrum Guide, essa abordagem serve para **otimizar a previsibilidade e controlar o risco**. Pode parecer contraditório que trabalhar sem um plano completo aumente a previsibilidade, mas pense bem: depois de algumas Sprints, a equipe tem dados reais sobre quanto consegue entregar e sobre o que os usuários valorizam. Esses dados permitem previsões muito mais confiáveis do que uma estimativa feita no escuro, antes de começar.

O **pensamento Lean** vem da indústria, especialmente do sistema de produção da Toyota, e tem como ideia central **reduzir desperdícios e focar no essencial**. Em software, desperdício pode ser uma funcionalidade que ninguém usa, um documento que ninguém lê, código que fica meses parado esperando ser testado, ou uma tarefa refeita porque alguém entendeu errado o que era para fazer.

Quando a equipe do aplicativo decide não construir agora um painel de estatísticas para o restaurante, porque ninguém sabe ainda quais números importam, ela está aplicando o pensamento Lean.

Por fim, o Scrum Guide observa que o Scrum envolve grupos de pessoas que, **coletivamente**, têm todas as habilidades e conhecimentos necessários para fazer o trabalho. Isso será importante quando falarmos do Scrum Team no próximo capítulo.

4.2 Os três pilares: transparência, inspeção e adaptação

O empirismo, na prática, se sustenta em três pilares: **transparência, inspeção e adaptação**. Eles funcionam como uma corrente, em que cada elo depende do anterior.

A **transparência** significa que o processo e o trabalho devem estar **visíveis** tanto para quem faz o trabalho quanto para quem o recebe. No Scrum, decisões importantes são tomadas com base no estado dos três artefatos formais (Product Backlog, Sprint Backlog e Increment, que veremos no Capítulo 7). Se esses artefatos não refletem a realidade, as decisões tomadas a partir deles serão ruins. O Scrum Guide diz que artefatos com baixa transparência levam a decisões que diminuem o valor e aumentam o risco.

Veja um exemplo de falta de transparência. Na equipe do aplicativo, um desenvolvedor marca a tela de pagamento como “pronta”, mas ela ainda não foi testada com cartões de débito. Todos acreditam que o pagamento está concluído, o Product Owner promete ao restaurante que o recurso estará disponível na semana seguinte e, no dia do lançamento, os pagamentos com débito falham. O problema não foi só técnico: foi de transparência. Se o estado real do trabalho estivesse visível, a decisão de prometer o lançamento teria sido outra. É por isso que o Scrum dá tanta importância a uma definição compartilhada do que significa “pronto”, a Definition of Done, que estudaremos adiante.

A **inspeção** significa examinar com frequência e atenção os artefatos e o progresso em direção aos objetivos combinados, para detectar problemas ou desvios indesejados. Para que a inspeção aconteça de forma regular, o Scrum oferece uma cadência, isto é, um ritmo previsível, por meio de seus cinco eventos. O Scrum Guide faz uma ligação importante aqui: **a transparência possibilita a inspeção**. Inspecionar algo que não está visível, ou que está representado de forma enganosa, é inútil e gera desperdício.

A **adaptação** significa ajustar o processo ou o produto quando a inspeção revela que algo saiu dos limites aceitáveis. O Scrum Guide diz que o ajuste deve ser feito **o mais rápido possível**, para minimizar novos desvios. E aqui aparece outra ligação: **a inspeção possibilita a adaptação**, e uma inspeção que não leva a nenhuma adaptação não tem sentido. De nada adianta a equipe perceber todas as semanas que os testes atrasam a entrega se nada for mudado.

O Scrum Guide acrescenta que a adaptação fica mais difícil quando as pessoas envolvidas **não têm autonomia** ou **não são autogerenciadas**. Se toda mudança precisar ser aprovada por três níveis de chefia, a equipe pode até perceber o problema, mas não vai conseguir reagir a tempo. Por isso se espera que o Scrum Team se adapte no momento em que aprende algo novo.

Uma forma simples de memorizar a relação entre os pilares é pensar num painel de carro. A transparência é o painel mostrar corretamente a velocidade e o combustível. A inspeção é o motorista

olhar para o painel com frequência. A adaptação é o motorista reduzir a velocidade ou parar para abastecer quando necessário. Sem um painel confiável, olhar para ele não adianta; olhar e não fazer nada também não.

4.3 Os cinco valores do Scrum

Os pilares explicam a mecânica do empirismo, mas há uma questão humana por trás: para que as pessoas sejam transparentes, inspecionem com honestidade e se adaptem, elas precisam de um ambiente de confiança. É aí que entram os **cinco valores do Scrum**, incluídos no Scrum Guide em 2016: **comprometimento, foco, abertura, respeito e coragem**.

O Scrum Guide explica cada valor de forma bem direta. Com **comprometimento**, o Scrum Team se compromete a alcançar seus objetivos e a apoiar uns aos outros. Repare que o compromisso é com os **objetivos** e com o **time**, não com a entrega de uma quantidade fixa de tarefas a qualquer custo. Com **foco**, a atenção principal está no trabalho da Sprint, para fazer o melhor progresso possível rumo aos objetivos. Com **abertura**, o time e as partes interessadas são abertos sobre o trabalho e os desafios. Com **respeito**, os membros do time se respeitam como pessoas capazes e independentes e são respeitados assim por quem trabalha com eles. Com **coragem**, os membros têm coragem de fazer a coisa certa e de enfrentar problemas difíceis.

Vamos ver como esses valores aparecem no dia a dia da equipe do aplicativo. Quando uma desenvolvedora percebe que a integração com o sistema de pagamento vai demorar muito mais do que o previsto, ela precisa de **coragem** e **abertura** para dizer isso imediatamente na reunião diária, em vez de esconder o problema e torcer para resolver sozinha. Os colegas, por sua vez, demonstram **respeito** ao tratar o aviso como uma informação útil, e não como uma falha pessoal, e demonstram **comprometimento** ao se oferecer para ajudar. Quando alguém de outra área pede para a equipe "só dar uma olhadinha" num relatório que não tem nada a ver com o objetivo da Sprint, o valor do **foco** ajuda a equipe a dizer que isso pode ser conversado com o Product Owner e considerado para depois.

O Scrum Guide observa que, quando esses valores são vividos pelo Scrum Team e pelas pessoas com quem ele trabalha, os pilares do empirismo ganham vida, construindo **confiança**. Perceba a conexão: sem abertura e coragem, não há transparência real; sem respeito, a inspeção vira caça às bruxas; sem comprometimento e foco, a adaptação não acontece. Os valores são, portanto, a base humana que permite que a teoria funcione. O guia também diz que as decisões tomadas, os passos dados e a forma de usar o Scrum devem reforçar esses valores, e não enfraquecê-los.

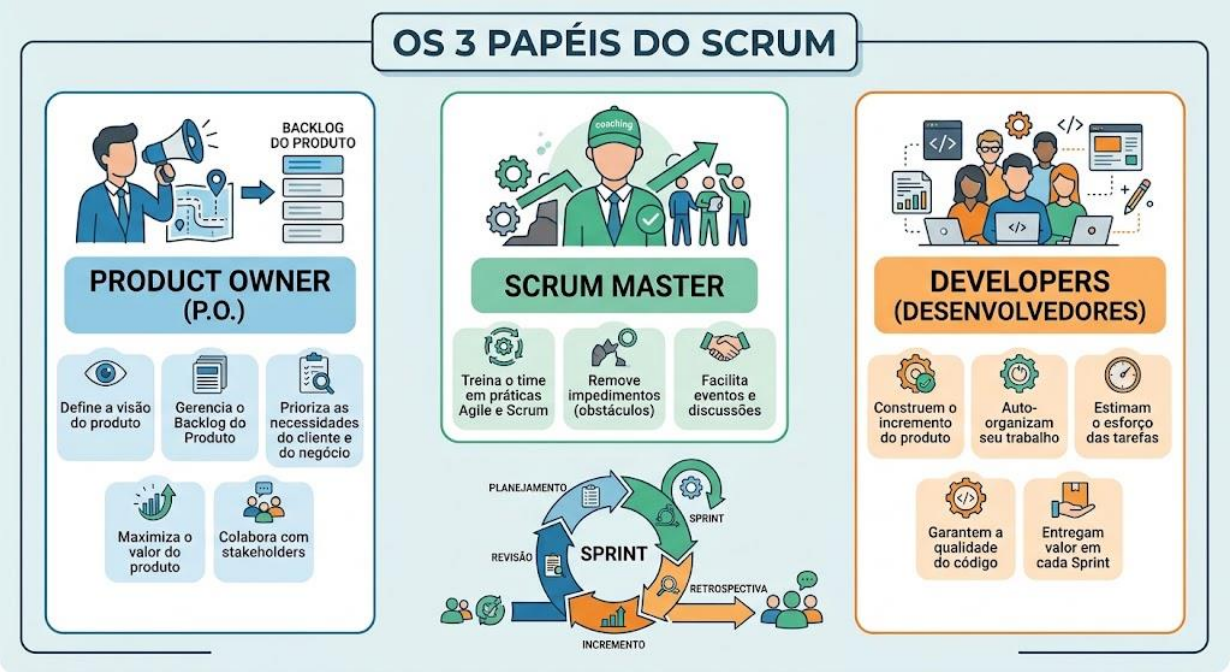
Com os fundamentos estabelecidos, podemos agora conhecer as pessoas que fazem o Scrum acontecer.

Fontes consultadas

- **Scrum Guides** – O Guia Definitivo do Scrum: As Regras do Jogo (novembro de 2020)

- **Scrum Alliance** – Guias e Recursos sobre o Framework Scrum
- **Agile Alliance** – Manifesto para Desenvolvimento Ágil de Software

4- Papéis do Scrum



A imagem apresenta um infográfico educativo focado na estrutura do **Scrum Team**, dividido em três colunas principais:

1. **Product Owner (P.O.):**
    a. **Função:** Define a visão do produto e maximiza o seu valor.
    b. **Atividades:** Gerencia e prioriza o *Product Backlog* e colabora diretamente com os *stakeholders*.

2. **Scrum Master:**
    a. **Função:** Garante a aplicação das práticas ágeis e atua como líder servidor.